

```
20211689@20211689-os ~]$ gcc hw6.c -lpthread
[20211689@20211689-os ~]$ ./a.out
Thread, made by Genie
Producer Write: 0 into 0-th buffer
Consumer Read: 0 from 0-th buffer
Producer Write: 1 into 1-th buffer
Consumer Read: 1 from 1-th buffer
Producer Write: 2 into 2-th buffer
Producer Write: 3 into 3-th buffer
Consumer Read: 2 from 2-th buffer
Producer Write: 4 into 0-th buffer
Producer Write: 5 into 1-th buffer
Consumer Read: 3 from 3-th buffer
Producer Write: 6 into 2-th buffer
Producer Write: 7 into 3-th buffer
Consumer Read: 4 from 0-th buffer
Producer Write: 8 into 0-th buffer
Producer Write: 9 into 1-th buffer
Consumer Read: 9 from 1-th buffer
Consumer Read: 6 from 2-th buffer
Consumer Read: 7 from 3-th buffer
Consumer Read: 8 from 0-th buffer
Consumer Read: -1 from 1-th buffer
[20211689@20211689-os ~]$
```

```
20211689@20211689-os ~]$ gcc hw6.c -lpthread
[20211689@20211689-os ~]$ ./a.out
Thread, made by Genie
Producer Write: 0 into 0-th buffer
Consumer Read: 0 from 0-th buffer
Producer Write: 1 into 1-th buffer
Consumer Read: 1 from 1-th buffer
Producer Write: 2 into 2-th buffer
Producer Write: 3 into 3-th buffer
Consumer Read: 2 from 2-th buffer
Producer Write: 4 into 0-th buffer
Producer Write: 5 into 1-th buffer
Consumer Read: 3 from 3-th buffer
Producer Write: 6 into 2-th buffer
Producer Write: 7 into 3-th buffer
Consumer Read: 4 from 0-th buffer
Producer Write: 8 into 0-th buffer
Consumer Read: 5 from 1-th buffer
Producer Write: 9 into 1-th buffer
Consumer Read: 6 from 2-th buffer
Consumer Read: 7 from 3-th buffer
Consumer Read: 8 from 0-th buffer
Consumer Read: 9 from 1-th buffer
[20211689@20211689-os ~]$
```

```
20211689@20211689-os:~$ cat hw6.c
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <time.h>

sem_t mutex, full, empty; // semaphore variable for mutual exclusion
int buffer[4] = {0, 0, 0, 0};

void *producer(void *arg) {
    int i;
    for(i=0; i<10; i++) {
        sem_wait(&full);
        sem_wait(&mutex); // Wait Operation of Semaphore
        printf("Consumer Read: %d from %d-th buffer\n", buffer[i % 4], i % 4);
        buffer[i % 4] = -1; // "-1" means "read & delete"
        sem_post(&mutex); // Signal Operation of Semaphore
        sem_post(&empty);
        sleep(2); // wait for two seconds
    }
}

void *consumer(void *arg) {
    int i;
    for(i=0; i<10; i++) {
        sem_wait(&empty);
        sem_wait(&mutex); // Wait Operation of Semaphore
        printf("Producer Write: %d into %d-th buffer\n", i, i % 4);
        buffer[i % 4] = i; // "i" means "write i-th item"
        sem_post(&mutex); // Signal Operation of Semaphore
        sem_post(&full);
        sleep(1); // wait for two seconds
    }
}

int main(void) {
    pthread_t id_p1, id_p2;

    printf("Thread, made by Genie\n"); // semaphore full = 0, empty = n, mutex = 1;
    sem_init(&mutex, 0, 1); // Initialize semaphore, mutex = 1
    sem_init(&empty, 0, 4); // empty = 4;
    sem_init(&full, 0, 0); // full = 0;

    pthread_create(&id_p1, NULL, producer, NULL);
    pthread_create(&id_p2, NULL, consumer, NULL);

    pthread_join(id_p1, NULL);
    pthread_join(id_p2, NULL);
}
```

기존 코드에서는 10, 9, -1이 출력되는 결과를 보였다. 전역변수 초기화 및 full 세마포어를 추가하였기 때문에 producer가 먼저 출력되고 0부터 출력된다. 또한, empty라는 세마포어를 추가한 이후엔 버퍼가 꽉 차게되면 producer에서 버퍼쓰기를 수행하지 않고 대기 상태가 되고, consumer에서 버퍼를 읽고 난 다음 signal을 보낼때 버퍼쓰기가 수행되도록 해서이다.